

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса
Шевченка ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
«Конструювання програмного забезпечення»
Лабораторна робота №3

Виконав здобувач:	Качанюк Дмитро	Перевірів:	Шабатін П.Є.
Група, підгрупа	ІПЗ-34, 10.	Дата перевірки	
2026			

Варіант №3: Інтерпретатор запитів до табличних даних (Micro-SQL Engine)

Тема: Захисне програмування, Обробка виключень та Модульне тестування.

Мета: Набути практичних навичок у:

- забезпеченні стійкості програмного забезпечення до некоректних вхідних даних;
- проектуванні та реалізації ієрархії власних виключень (Custom Exceptions);
- написанні модульних тестів (Unit Tests) з використанням сучасних фреймворків;
- використанні Mock-об'єктів для ізоляції тестованих компонентів; вимірюванні та контролі покриття коду тестами (Code Coverage).

Етап 1: Захисне програмування (Error Handling)

1.1 Ієрархія виключень

Для реалізації уніфікованого оброблення помилок створено власні класи виключень у файлі `src/exceptions.py`, що успадковуються від базового класу `Exception`:

```
class MicroSqlException(Exception):  
  
    def __init__(self, message: str, line: int = None):  
        self.line = line  
        prefix = f" at line {line}" if line else ""  
        super().__init__(f"{message}{prefix}")
```

```
class FileSystemException(MicroSqlException):  
    pass
```

```
class ParserException(MicroSqlException):  
    pass
```

```
class ValidationException(MicroSqlException):  
    pass
```

```
class TypeConflictException(ValidationException):  
    pass
```

Реалізовано наступні виключення:

MicroSqlException – базова помилка для всіх помилок системи. Використання як батьківського класу дозволяє перехоплювати будь-яку специфічну помилку одним блоком except, що відповідає принципу DRY.

FileSystemException – піднімається коли файл не знайдено або немає прав доступу до нього.

ParserException – піднімається коли SQL-запит містить синтаксичну помилку: відсутнє ключове слово SELECT або FROM, некоректна умова WHERE.

ValidationException — піднімається коли запит звертається до неіснуючої колонки у SELECT або WHERE.

TypeConflictException – успадковується від ValidationException, піднімається при спробі використати числовий оператор (>, <, >=, <=) з нечисловим значенням.

1.2 Рівні перехоплення помилок

Програму спроектовано за принципом «Fail Fast» та централізованого перехоплення. Кожен модуль відповідає за валідацію власних інваріантів:

QueryParser: перевіряє наявність ключових слів SELECT та FROM, коректність умов WHERE. При виявленні помилки піднімає ParserException з вказівкою номера рядка.

CsvLoader: перевіряє існування файлу та наявність заголовків. При відсутності файлу піднімає FileSystemException.

Executor: перед виконанням запиту перевіряє наявність усіх колонок із SELECT та WHERE у таблиці. При конфлікті типів у порівнянні піднімає TypeConflictException.

Проміжні шари не перехоплюють помилки всередині себе – виключення піднімаються до найвищого рівня. Централізована обробка зосереджена в cli.py:

try:

```
query = query_parser.parse(sql_file_path)
table = csv_loader.load(query.table_name)
result_rows = executor.execute(table, query)
formatter.print_results(result_rows)
```

except MicroSqlException as error:

```
error_type = type(error).__name__
print(f'Error: [{error_type}] - {error}')
sys.exit(1)
```

Завдяки тому, що всі кастомні класи успадковуються від MicroSqlException, один блок except перехоплює будь-яку специфічну помилку системи та виводить її у форматі:

Error: [ParserException] - Missing keyword FROM in query at line 1
Error: [ValidationException] - Column 'age' not found in table at line 2
Error: [TypeConflictException] - Cannot use '>' to compare 'John' and 'high'
Error: [FileSystemException] - File not found: 'data/missing.csv'

Етап 2: Модульне тестування (Unit Testing)

2.1 Фреймворк та інструменти

Для написання тестів обрано бібліотеку `pytest` – одну з найпоширеніших для Python, яка має зручний функціонал для фікстур, мокування та різних видів assertions. Для вимірювання покриття використано `pytest-cov` (Coverage.py).

Команда запуску тестів:

`pytest`

Команда з покриттям:

`pytest --cov=src --cov-report=term-missing`

2.2 Структура тестів

Тести розміщені у папці `tests/` та розподілені по трьох файлах відповідно до модулів системи:

Файл	Модуль	Кількість тестів
<code>tests/test_parser.py</code>	<code>QueryParser</code>	10
<code>tests/test_engine.py</code>	<code>Executor</code>	9
<code>tests/test_csv_loader.py</code>	<code>CsvLoader</code>	5
Разом		24

2.3 Тестові сценарії

Для кожного модуля реалізовано позитивні та негативні тести.

QueryParser – tests/test_parser.py:

Позитивні тести:

`test_parsing_select_columns` – перевірка коректного розбору списку колонок

`test_parsing_table_name` – перевірка отримання імені таблиці з FROM

`test_parsing_where_condition` – перевірка розбору одиночної умови WHERE

`test_parsing_multiple_conditions` – перевірка кількох умов через AND

`test_parsing_order_by_desc` – перевірка ORDER BY з напрямком DESC

`test_parsing_star_select` – перевірка SELECT *

test_no_where_returns_empty_conditions — перевірка відсутності WHERE

Негативні тести:

test_missing_select_raises_parser_exception – відсутнє SELECT → ParserException

test_missing_from_raises_parser_exception – відсутнє FROM → ParserException

test_invalid_condition_raises_parser_exception – некоректна умова → ParserException

Executor — tests/test_engine.py:

Позитивні тести:

test_filters_by_department – фільтрація за одним рядковим полем

test_filters_by_multiple_conditions – фільтрація за двома умовами (AND)

test_sorts_by_salary_desc — сортування за числовим полем DESC

test_select_projects_columns – перевірка проєкції колонок

test_select_star_returns_all_columns – SELECT * повертає всі колонки

test_no_results_returns_empty_list – запит без результатів

Негативні тести:

test_unknown_select_column_raises_validation – неіснуюча колонка у SELECT → ValidationException

test_unknown_where_column_raises_validation – неіснуюча колонка у WHERE → ValidationException

test_type_conflict_raises_exception – порівняння рядка числовим оператором → TypeConflictException

CsvLoader — tests/test_csv_loader.py:

Позитивні тести:

test_loads_headers – перевірка коректного зчитування заголовків

test_loads_correct_row_count – перевірка кількості рядків

test_row_values_accessible – перевірка доступу до значень через Row.get()

Негативні тести:

test_missing_file_raises_filesystem_exception – відсутній файл → FileSystemException

test_empty_file_raises_filesystem_exception – порожній CSV → FileSystemException

2.4 Mock-об'єкти

При тестуванні QueryParser та CsvLoader використано mock_open з бібліотеки unittest.mock для ізоляції від реальної файлової системи. Це дозволяє тестувати логіку модулів незалежно від наявності файлів на диску:

```
from unittest.mock import mock_open, patch
```

```
def test_parsers_select_columns(self, parser):
    sql = "SELECT name, salary FROM data/users.csv"
    with mock_sql(sql):
        ast = parser.parse("query.sql")
        assert ast.columns == ["name", "salary"]
```

При тестуванні Executor використано фікстури pytest з реальними об'єктами Row та Table, що дозволило ізолювати логіку виконання запитів від парсингу та завантаження файлів.

2.5 Результати тестування

Всі 24 тести пройшли успішно:

```
(.venv) kachanyukdima@MacBook-Air-Kacanuk micro_sql_engine % pytest
===== test session starts =====
platform darwin -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/kachanyukdima/Desktop/Univer/3course/2sem/Software_Design/micro_sql_engine
configfile: pytest.ini
testpaths: tests
plugins: cov-7.1.0
collected 24 items

tests/test_csv_loader.py .....
tests/test_engine.py .....
tests/test_parser.py .....
```

2.6 Покриття коду (Code Coverage)

Після запуску pytest --cov=src --cov-report=term-missing отримано наступні результати:

Name	Stmts	Miss	Cover	Missing
src/engine/__init__.py	0	0	100%	
src/engine/executor.py	71	8	89%	47, 65, 74-82, 89-91, 103
src/exceptions.py	13	0	100%	
src/model/__init__.py	0	0	100%	
src/model/expression_tree.py	12	0	100%	
src/model/table.py	23	2	91%	13, 25
src/parser/__init__.py	0	0	100%	
src/parser/query_parser.py	74	5	93%	52, 60, 65, 103-104
src/utils/__init__.py	0	0	100%	
src/utils/csv_loader.py	18	1	94%	31
src/utils/output_formatter.py	25	25	0%	1-32
TOTAL	236	41	83%	

Загальне покриття коду становить 83%, що перевищує мінімальну вимогу у 70%.

Висновок

У ході роботи реалізовано систему захисного програмування для Micro-SQL Engine на базі ієрархії власних виключень MicroSqlException. Програма більше не

завершується аварійно при некоректних вхідних даних – всі помилки перехоплюються централізовано в cli.py та виводяться у форматovanому вигляді Error: [ТипПомилки] - Опис.

Кожен модуль системи самостійно валідує вхідні дані: QueryParser перевіряє синтаксис SQL-запиту, CsvLoader – існування та коректність файлу, Executor – відповідність колонок та сумісність типів при порівнянні.

Завдяки використанню фреймворку pytest та Mock-об'єктів реалізовано ізольоване тестування бізнес-логіки без залежності від реальних файлів. Написано 24 тести (17 позитивних та 7 негативних), усі пройшли успішно. Покриття коду становить 83%, що підтверджує надійність розробленої архітектури та її повну відповідність технічним вимогам лабораторної роботи.